

→ D. arrays are an improvement based on the static ones, what changes is that now this D.S keeps a pointer to the "Next sequential empty index" in it.

↳ And also D. arrays can automatically increase its size whenever out of space to allocate new items.

↳ this specific feature runs in $O(N)$ because a bigger array would have to be created and then the current array would have to have its values (in same positions) copied to the new version, meaning you would have to traverse all items from Source array anyway.

→ As you can see adding items to the array in the worst case scenario (Resizing) costs $O(N)$. But in most of the cases (when you

$O(1)$, and in most of the cases, when array has spare room for new items) it runs on $O(1)$.

↳ because of this peculiarity, D. arrays have an "amortized complexity" for adding new items of $O(1)$, because it will be the case in most of the times.